

ПОЛНЫЙ ТЕХНИЧЕСКИЙ БЛУПРИНТ АВТОМАТИЗАЦИИ

Разработка экосистемы онлайн-бронирования (Telegram Mini App + Go + PostgreSQL + bePaid)
для базы отдыха "Олтуш"

Настоящий документ представляет собой исчерпывающее техническое руководство и спецификацию архитектуры для создания независимой SaaS-системы управления и бронирования объектов недвижимости. Система ориентирована на максимальную автоматизацию бизнес-процессов усадьбы (8 домиков, баня, купель), минимизацию человеческого фактора и исключение овербукинга.

1. Общая архитектура и стек технологий

Система проектируется как монолитное или легко-сервисное приложение, разделенное на три логических слоя с четкими границами ответственности. Отказ от конструкторов и сторонних платформ обеспечивает максимальную скорость работы и независимость от лицензий.

Бэкенд (Go)

Высокопроизводительный сервер на Go (чистый REST API, библиотека `chi` или `gin`). Обработывает бизнес-логику, конкурентные транзакции, интеграцию с API

мессенджера и платежные сервисы.

Фронтенд (React)

Telegram Mini App (TMA), написанный на React + Tailwind CSS. Обеспечивает плавный нативный интерфейс (карты, календари) внутри мессенджера без перезагрузок страниц.

База данных (PostgreSQL)

Реляционная СУБД для надежного хранения данных. Строгие внешние ключи, транзакции уровня изоляции `Serializable` или блокировки строк предотвращают овербукинг.

2. Проектирование базы данных (PostgreSQL Схема)

Для обеспечения целостности данных, учета занятости объектов и отслеживания транзакций создается структура из пяти базовых таблиц. Использование UUID и четких ограничений гарантирует масштабируемость системы.

```
-- Расширение для генерации UUID, если потребуется уникальная идентификация заказов
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";

-- Таблица объектов базы отдыха
CREATE TABLE items (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  type VARCHAR(50) NOT NULL, -- 'house' (домик), 'sauna' (баня), 'tub' (купель)
  price_per_night NUMERIC(10, 2) NOT NULL,
  max_guests INT NOT NULL DEFAULT 4,
  is_active BOOLEAN DEFAULT TRUE
);

-- Таблица пользователей (клиентов) из Telegram
CREATE TABLE customers (
  id SERIAL PRIMARY KEY,
  telegram_id BIGINT UNIQUE NOT NULL,
  username VARCHAR(100),
  first_name VARCHAR(100),
  phone VARCHAR(20) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Таблица бронирований (основной реестр занятости)
CREATE TABLE bookings (
  id SERIAL PRIMARY KEY,
  item_id INT REFERENCES items(id) ON DELETE RESTRICT,
  customer_id INT REFERENCES customers(id) ON DELETE RESTRICT,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  status VARCHAR(30) NOT NULL DEFAULT 'pending', -- 'pending', 'confirmed', 'cancelled'
  total_price NUMERIC(10, 2) NOT NULL,
  is_manual_override BOOLEAN DEFAULT FALSE, -- признак ручного внесения админом
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT check_dates CHECK (start_date < end_date)
);

-- Таблица транзакций оплаты через платежный шлюз
CREATE TABLE payments (
  id SERIAL PRIMARY KEY,
  booking_id INT REFERENCES bookings(id) ON DELETE CASCADE,
  bepaid_uid VARCHAR(100) UNIQUE, -- Уникальный ID транзакции в bePaid
  amount NUMERIC(10, 2) NOT NULL,
  status VARCHAR(30) NOT NULL, -- 'successful', 'failed', 'incomplete'
  payment_type VARCHAR(20), -- 'card', 'erip'
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

⚠ Защита от race conditions (одновременного бронирования)

Для предотвращения ситуации, когда два пользователя одновременно пытаются забронировать один домик на пересекающиеся даты, на бэкенде в рамках одной транзакции выполняется строгая блокировка строки и проверка занятости:

```
SELECT COUNT(*) FROM bookings WHERE item_id = $1 AND status IN ('pending',
'confirmed') AND NOT (end_date <= $2 OR start_date >= $3) FOR UPDATE;
```

3. Полный функционал и сценарии работы бота

Система делит пользователей на два лагеря: Клиенты (внешний Mini App) и Администрация (внутренний интерфейс и триггерные уведомления).

3.1. Клиентский путь (Пошаговый алгоритм)

- 1. Старт и авторизация:** Пользователь нажимает /start в боте. Бот проверяет наличие пользователя в таблице customers. Если его нет, бот отправляет приветствие и кнопку-запрос на отправку контакта (share native phone number в Telegram). Это исключает фейковые номера.
- 2. Вход в Mini App:** После верификации телефона появляется инлайн-кнопка «Забронировать отдых», открывающая Web App.
- 3. Выбор объекта на интерактивной карте:** На главном экране Mini App отображается SVG-схема территории (8 домиков, баня на берегу, купель). Объекты интерактивны: свободные подсвечены зеленым контуром, занятые в текущий день — оранжевым. При тапе на объект открывается карточка (фото, характеристики, вместимость).
- 4. Календарь занятости:** При нажатии на кнопку «Выбрать даты» открывается интерактивный календарь. Фронтенд шлет запрос на бэкенд и получает список дат, где объект уже забронирован в статусе confirmed или pending. Эти дни блокируются. Пользователь кликает на дату заезда и дату выезда.

5. **Расчет стоимости и выбор типа оплаты:** Система мгновенно калькулирует стоимость по формуле: $S = N \times P$, где N — количество ночей, а P — базовая стоимость за сутки. Пользователю предлагаются два варианта: Внести предоплату 100% или Предоплату за 1 сутки (страховка от отмены).

3.2. Административная CRM-панель (Для владельцев базы)

Админы получают доступ к скрытому Mini App (защищенному по проверке telegram_id из списка разрешенных в конфигурационном файле бэкенда).

Модуль админки	Технический функционал	Ценность для бизнеса
Интерактивная Шахматка	Календарная сетка (Ось Y — домики, Ось X — дни месяца). Рендерится на фронтенде в виде горизонтальных цветных полос-записей. Показывает текущую загрузку базы.	Визуальный контроль заездов и выездов в реальном времени, заменяет бумажные журналы.
Ручной оверрайд (Внесение брони)	Форма создания записи в обход платежного шлюза. Админ вводит даты, имя и телефон гостя (например, если тот позвонил на мобильный). Запись сразу пишется в БД со статусом confirmed.	Позволяет принимать классические звонки по телефону, полностью исключая двойное заселение.
Управление тарифами	Эндпоинты для быстрого изменения цен (PUT /api/admin/items/:id). Возможность заблокировать домик на определенные даты, переведя его в статус "Ремонт".	Гибкое управление прибылью в праздничные дни и высокий сезон без изменения кода.

4. Интеграция с платежной системой bePaid (Беларусь)

Для легального приема платежей от физлиц в Республике Беларусь для ИП используется интеграция с агрегатором bePaid через API. Форма ввода данных банковской карты никогда не рендерится внутри ТГ-бота — это критически важное требование безопасности.

Последовательность обработки платежа и логика стейт-машины

Когда клиент нажимает кнопку «Оплатить», бэкенд инициирует сессию бронирования:

- В БД создается запись в таблице bookings со статусом pending. С этого момента даты заблокированы в календаре ровно на 15 минут.
- Бэкенд формирует POST-запрос к API bePaid на создание платежной страницы (Checkout). В запросе передается ID магазина, секретный ключ, сумма заказа в BYN, описание и, самое главное, notification_url — адрес нашего вебхука.
- bePaid возвращает JSON, содержащий уникальный token и длинный URL вида `https://checkout.bepaid.by/v2/checkout?token=...`. Бэкенд сохраняет токен в таблицу payments и отдает URL на фронтенд.
- Mini App открывает эту ссылку во встроенном браузере Telegram (In-App Browser). Пользователь видит защищенный шлюз, вводит карту или выбирает оплату через ЕРИП.

5. **Сценарий А (Успех):** Деньги списаны. bePaid посылает асинхронный POST-запрос (вебхук) на бэкэнд бота. Бэкэнд проверяет цифровую подпись запроса (Basic Auth), парсит JSON, находит нужную бронь, переводит ее статус в `confirmed`, а платеж — в `successful`. Бот отправляет уведомления клиенту и админу.
6. **Сценарий Б (Отказ/Таймаут):** Если в течение 15 минут вебхук об успешной оплате не пришел, фоновый воркер (cron-задача) на бэкэнде удаляет устаревшие записи со статусом `pending`, освобождая даты в календаре для других клиентов.

```
// Пример структуры JSON, отправляемой бэкэндом на bePaid API для генерации счета
{
  "checkout": {
    "version": "2.1",
    "transaction_type": "payment",
    "order": {
      "currency": "BYN",
      "amount": 15000,
      "description": "Предоплата за бронирование: Домик №1, 05.06 - 06.06"
    },
    "settings": {
      "success_url": "https://t.me/OltushBot/app?startapp=success",
      "decline_url": "https://t.me/OltushBot/app?startapp=fail",
      "notification_url": "https://api.bazaoltush-bot.by/v1/webhook/bepaid"
    }
  }
}
```

5. План развертывания и модель монетизации (SaaS)

Для обеспечения бесперебойной работы системы и создания стабильного источника пассивного дохода для разработчика внедряется арендная модель обслуживания софта.

5.1. Инфраструктура хостинга

- **Виртуальный сервер (VPS):** Достаточно минимального тарифа у белорусских хостеров (HostFly.by, Hoster.by) или стабильного PQ.hosting за \$4–\$5 в месяц (1 core CPU, 2GB RAM, 20GB SSD, Ubuntu Server).
- **Контейнеризация:** Весь проект упаковывается в Docker-compose (один контейнер — Go бэкэнд, второй — база данных PostgreSQL, третий — веб-сервер Nginx для проксирования запросов и отдачи статики React-приложения). Это гарантирует, что проект запустится на любом сервере в один клик.
- **Безопасность:** Через утилиту certbot выпускается бесплатный SSL-сертификат Let's Encrypt для домена API. Без HTTPS-протокола bePaid категорически откажется присылать вебхуки.

5.2. Пассивный доход и экономика поддержки

Продукт продается владельцам по комбинированной схеме: разовая оплата за создание кода и настройку под ключ (\$400–\$500) + ежемесячная абонентская плата за администрирование (\$25–\$35 или 80–110 BYN).

Что входит в ежемесячную поддержку: Мониторинг работоспособности Docker-контейнеров, оплата самого VPS-сервера, продление доменного имени, ротация логов (чтобы сервер не забился мусором) и ежедневный автоматический бэкап базы данных Postgres в облачное хранилище (например, Yandex Cloud Object Storage или Telegram-канал архивов) по расписанию через простой Bash-скрипт.



Твоя страховка как разработчика

Поскольку бэкэнд и база данных развернуты на сервере, доступ к которому контролируешь ты, бизнес физически зависит от твоей поддержки. Если клиент без предупреждения прекращает оплачивать ежемесячную подписку, ты выполняешь одну команду `docker-compose down` в терминале. Вся система онлайн-бронирования мгновенно останавливается, защищая твою интеллектуальную собственность.

6. Пошаговый план промптинга для ИИ-агентов (Windsurf / Cursor)

Чтобы собрать эту систему быстро и без багов, двигайся строго по этапам, скармливая ИИ-агенту изолированные задачи. Не проси его написать всё сразу.

Этап	Что скормить ИИ-агенту (Промпт)	Ожидаемый результат этапа
Этап 1: БД и Бэкэнд	"Сгенерируй структуру таблиц PostgreSQL для системы бронирования домиков. Напиши бэкэнд на Go с использованием роутера chi. Реализуй эндпоинты GET /api/items и GET /api/bookings/busy?item_id=X&month=Y, который возвращает массив дат, заблокированных для бронирования."	Готовый SQL-скрипт инициализации БД, модели данных в Go, логика подключения к Postgres через pgx и рабочие эндпоинты для отдачи календаря занятости.
Этап 2: Интеграция bePaid	"Добавь в бэкэнд на Go модуль для работы с API bePaid Sandbox. Реализуй эндпоинт POST /api/bookings/create, который создает запись со статусом pending и запрашивает ссылку на оплату у bePaid. Напиши обработчик POST /api/webhook/bepaid, который проверяет авторизацию и переводит статус брони в confirmed."	Логика стейт-машины платежей, генерация платежных ссылок песочницы bePaid, безопасная обработка вебхуков и перевод статуса заказов при успешной оплате.

Этап	Что скормить ИИ-агенту (Промпт)	Ожидаемый результат этапа
Этап 3: Календарь (React)	"Создай фронтенд на React и Tailwind CSS для Telegram Mini App. Напиши компонент интерактивного календаря. Компонент должен делать запрос к GET /api/bookings/busy, получать массив занятых строк-дат и полностью блокировать их для клика пользователя. При выборе свободных дат подсвечивай диапазон зеленым."	Визуальный модуль календаря, динамически реагирующий на занятость домиков, исключающий человеческую ошибку при выборе дней заезда на фронтенде.
Этап 4: Админка	"Напиши защищенный роут на фронтенде для админов базы отдыха. Реализуй экран 'Шахматка', отображающий сетку домиков и заездов на текущий месяц. Добавь форму для ручного создания бронирования (дата заезда, выезда, имя, телефон), которая шлет запрос на POST /api/admin/bookings/manual."	Полноценное CRM-приложение для ведения ручного учета гостей, управления ценами и полного визуального контроля над объектами усадьбы админом.

Документ является официальным руководством по разработке архитектуры системы бронирования
v1.0.